

CodeFabInterpreter

C++ 커스텀 언어 인터프리터 개발

2026. 06.

hahagil · epicpoem · mingusim · jiyun7-Lee

목차

01

프로젝트 소개

02

인터프리터 파이프라인

03

시연

04

개발 **Best Practice**

05

회고



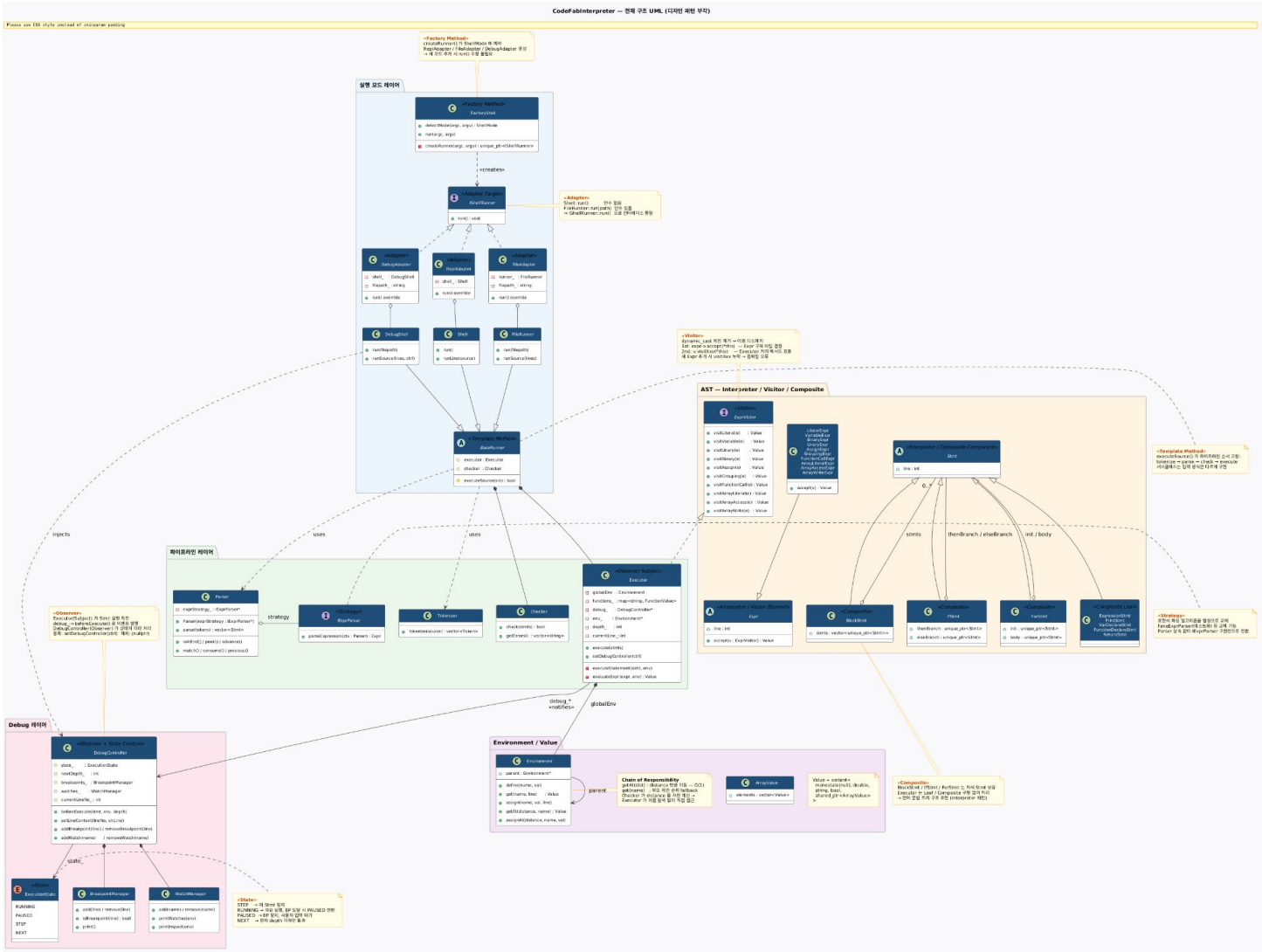
01 프로젝트 소개



팀 구성 & 역할 분담

송용길 (hahagil)		김홍원 (epicpoem)		심민구 (mingusim)		이지윤 (jiyun7-Lee)	
1차	1차: Tokenizer	1차	1차: Expression Parser	1차	1차: Statement Parser + Checker	1차	1차: Executor + Shell
2차	2차: 추가 연산자 구현 Negative UT 31개 추가	2차	2차: Function 구현 Array 구현	2차	2차: Static Binding Constant Folding	2차	2차: FileRunner / FactoryShell Debug Mode (BP/Watch/Step)

전체 구조 UML



[link](#)

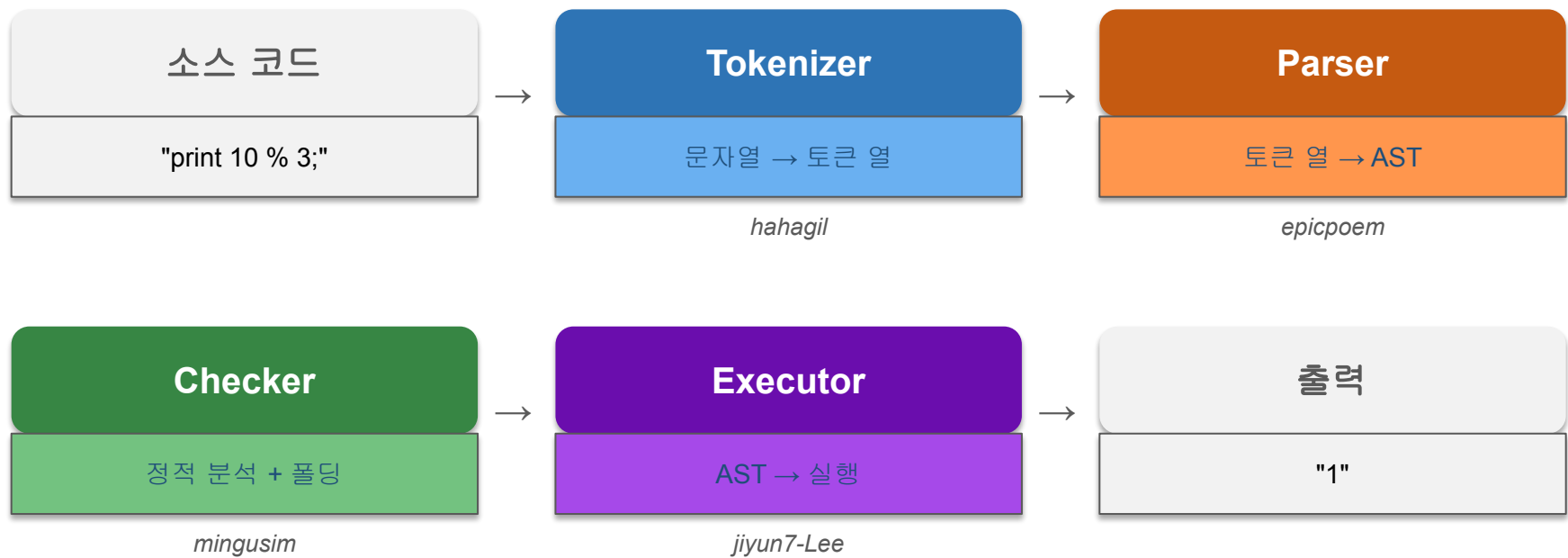


02 인터프리터 파이프라인



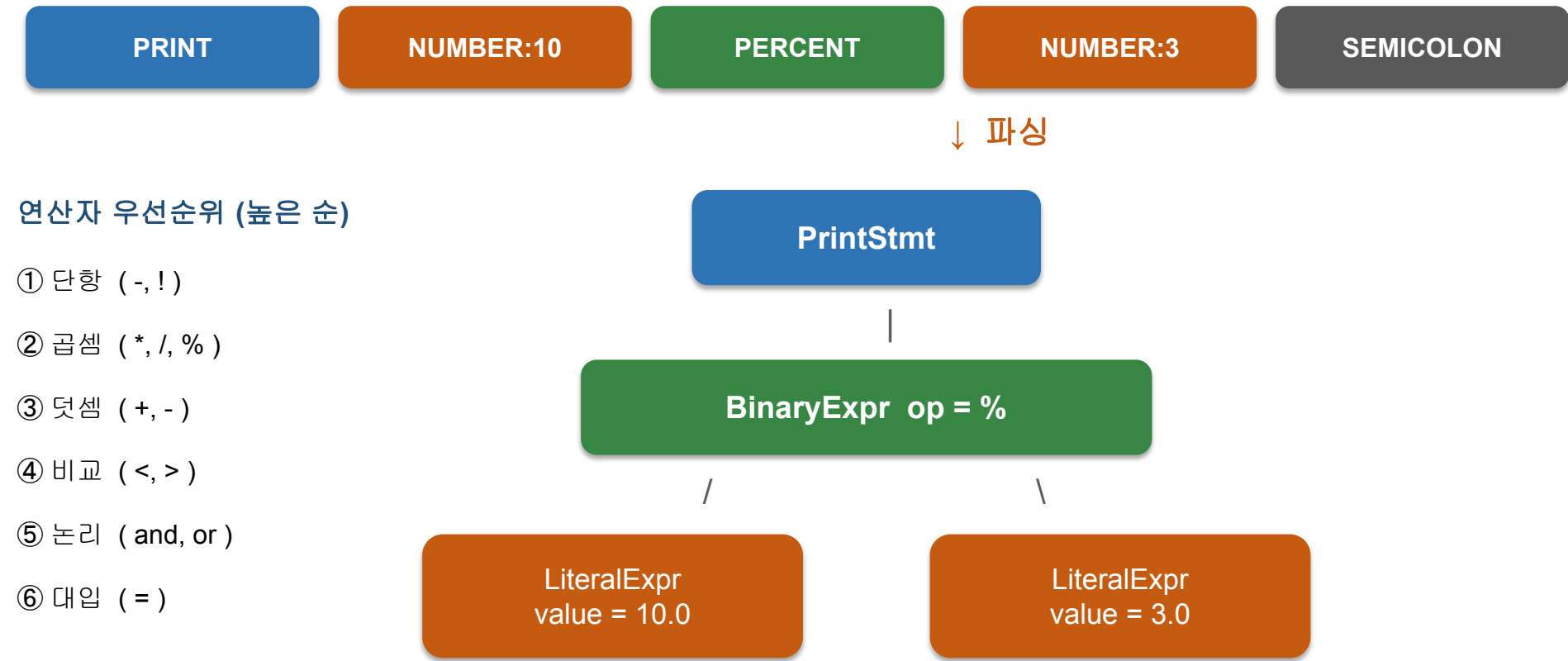
파이프라인 전체 흐름

BaseRunner.executeSource() — tokenize → parse → check → execute 공통 파이프라인



Parser — 토큰 열 → Abstract Syntax Tree(AST)

재귀 하강 파싱으로 토큰 열을 Abstract Syntax Tree(AST)로 구성



Checker — 정적 분석 + 최적화

정적 분석

미선언 변수 참조

`print x;` → `x`가 선언되지 않음 에러

자기 참조 초기화

`var a = a;` → 선언 중 자기 참조
에러

함수 인자 수 불일치

`func f(a,b)` → `f(1)` 호출 시 에러

Static Binding

- `VariableExpr` 에 스코프 `distance` 기록
- `Executor`가 이름 탐색 없이 `distance` 만큼 이동

`distance = 0`

→ `getAt(0, 'x')` 현재 스코프 직접 접근

Constant Folding

`2 + 3`

→ `Lit(5)` ← 런타임 연산 없음

`0 + x`

→ `x` (대수 항등식)

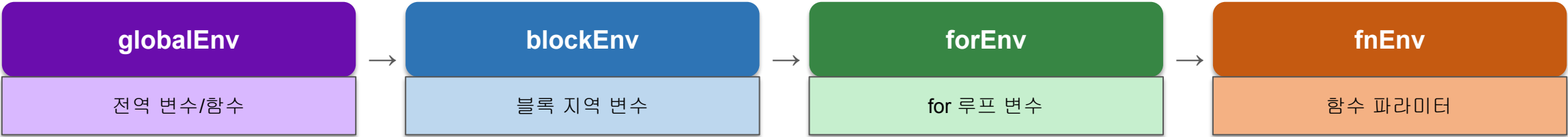
`9 % 0`

✗ 폴딩 안 함 (0 나누기)

Executor — AST 실행

Checker를 통과한 AST를 순회하며 실제 값 계산·출력

환경 (Environment) 체인



$distance \geq 0 \rightarrow getAt(d, name)$ 이름 탐색 없이 d 단계 직접 이동 | $-1 \rightarrow$ 체인 순회 *fallback*

주요 처리

산술/비교/논리	PLUS/MINUS/STAR/SLASH/PERCENT LESS/GREATER AND/OR
문자열 연결	PLUS: string+string $\rightarrow$ "Hello" + "World" = "HelloWorld"
함수 호출	fnEnv 생성 $\rightarrow$ ReturnSignal 예외로 반환값 전달
배열	Array(n) 빌트인 arr[i] 읽기/쓰기 인덱스 범위 검사

Debug Mode — DebugController

Executor 실행 흐름에 훅(hook)을 걸어 줄별 정지·변수 감시·breakpoint를 제공한다

ExecutionState

RUNNING
자유 실행

STEP
줄별 정지

NEXT
블록 skip

PAUSED
breakpoint 정지

BreakpointManager

- add(line) / remove(line) / isBreakpoint(line)
- RUNNING 상태에서 줄번호 일치 시 PAUSED 전환

WatchManager

- add(name) / remove(name)
- 정지 시 감시 변수 값 자동 출력

Debug 명령어

step / s

줄 실행 후 정지

next / n

블록 진입 없이 이동

continue / c

breakpoint까지 실행

break N

N번째 줄 breakpoint

watch V

변수 V 감시 등록



03 시연



시연 순서

①	기본 연산	<code>print 10 % 3;</code>
②	문자열 연결	<code>print "Hello, " + "World!";</code>
③	변수 + 조건문	<code>var x = 10; if (x > 5) { ... }</code>
④	for 반복문	<code>for (var i=0; i<5; i=i+1) { ... }</code>
⑤	함수 + 재귀	<code>func factorial(n) { ... } → 120</code>
⑥	배열	<code>var arr = Array(3); arr[0] = 10;</code>
⑦	에러 처리	<code>print 10 / 0; → [Error] 0으로 나눈 오류</code>
⑧	Debug 모드	<code>debug sample.cb → step / break / watch</code> 시연



04 개발 Best Practice



BP 1 — TDD (Test-Driven Development)

테스트를 먼저 작성하고, 구현이 따라오게 한다 — 348개 TC, 레이어 간 통합 충돌 0건

BP 사례 ① — % 연산자 구현 (Issue #27)

- [test]** 4개 레이어(Tokenizer/Parser/Executor/Optimizer) TC 동시 작성 → 전부 Red
- [feat]** Tokenizer PERCENT 토큰 추가 → Parser 파싱 → Executor fmod 실행 → Checker 폴딩
- [refactor]** Executor SLASH/PERCENT 중복 0-검사 단일 블록으로 통합

BP 사례 ② — 이슈 기반 TC 추가 (Issues #28~#33 → PR #37)

- 이슈 #28~#33: 7개 모듈에서 누락된 TC를 이슈로 등록
- TC-FN-16(함수 외부 return 오류 메시지에 줄번호 포함)이 Red → Stmt.h · Parser.cpp · Checker.cpp 3파일 수정 유발
- 이슈가 실행 가능한 TC 명세로 변환 → 구현이 자동으로 따라오는 구조

Issue-Driven 기능 추가 & 버그 수정

#27	% 연산자	기능 없음 → TC Red → 파이프라인 전 레이어 구현	#28~#33	누락 TC 14개	이슈 등록 → TC 작성 → 프로덕션 코드 3파일 수정	#42	REPL { 오입력	멀티라인 brace depth 오계산 → Tokenizer 기반 재구현
-----	-------	---------------------------------------	---------	-----------	--------------------------------------	-----	------------	---

BP 2 — PR 기반 코드 리뷰

모든 변경은 PR로 제출 → 최소 1인 리뷰 → 승인 후 master merge

리뷰 사례 1 — Checker.cpp <cmath> 누락 (PR #39, epicpoem 리뷰)

- Checker.cpp에서 `std::fmod` 사용 중 `#include <cmath>` 미포함 → MSVC 우연 빌드, GCC/Clang 컴파일 오류 가능
- 즉시 amend → <cmath> 추가 후 force-push → 재검토 완료

[link](#)

리뷰 사례 2 — FileRunner 상태 유실 (PR #35, hahagil 리뷰)

- 줄별 독립 Executor 생성 → 앞 줄 변수가 다음 줄에서 소멸 → Critical 지적
- Executor / Checker를 FileRunner 멤버 변수로 승격 → 상태 유지 보장 → 재승인

[link](#)

BP 3 — AI (Claude Code) 활용

llvm-cov 커버리지 측정 자동화 — 환경 셋업부터 one-click 실행까지

문제

OpenCppCoverage → line coverage만 제공

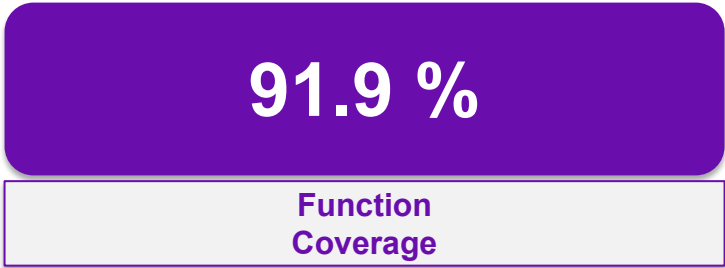
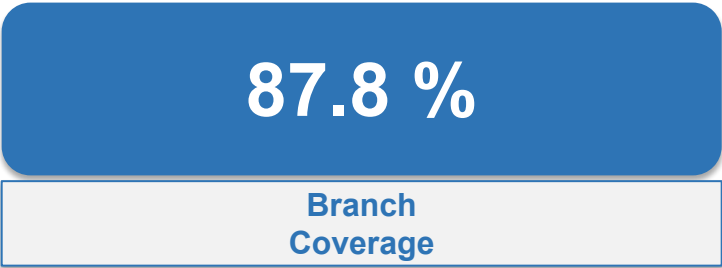
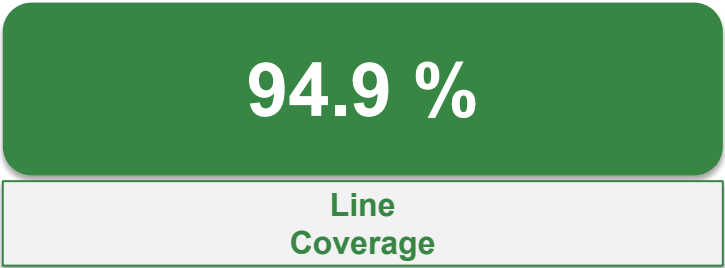
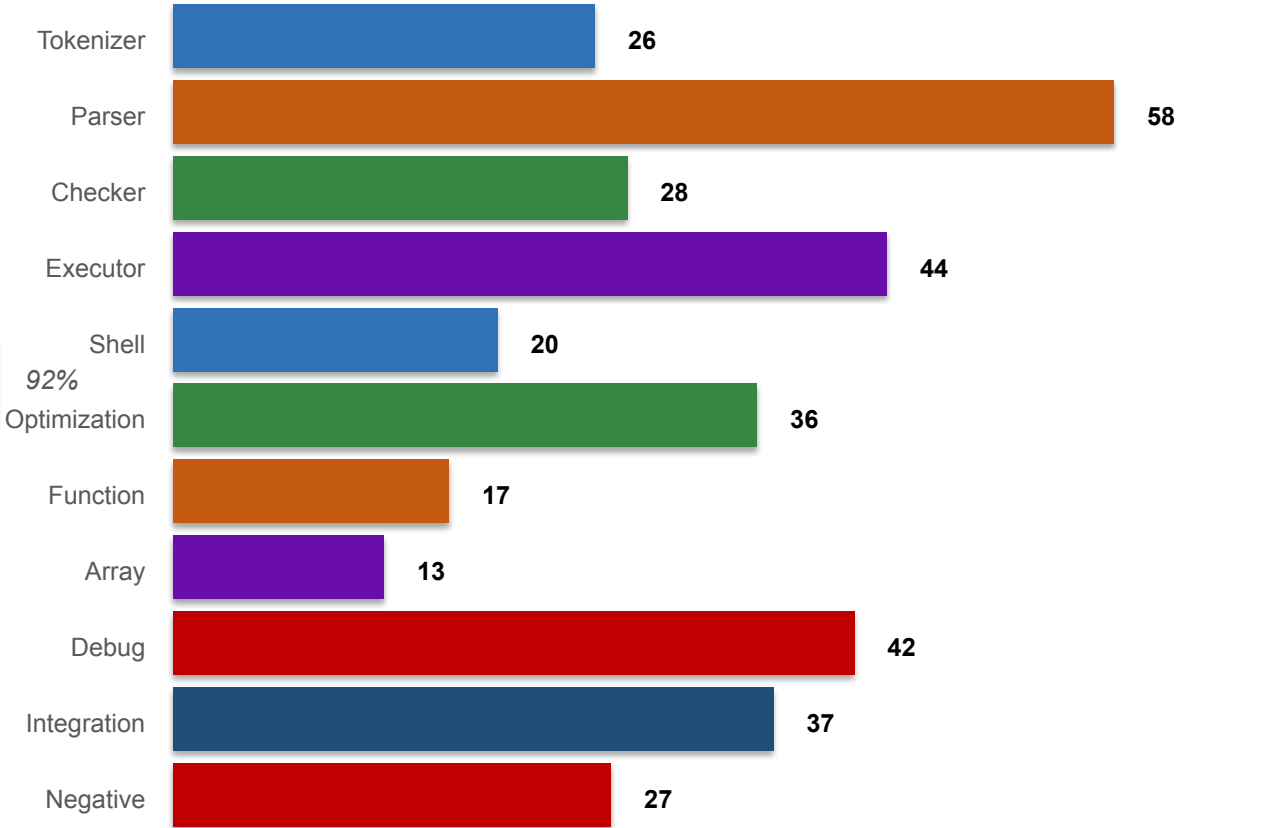
- function 단위·branch 단위 coverage를 측정할 수 없어 테스트 품질의 취약 지점 파악이 어려웠음
- llvm-cov 도입 시 컴파일러 플래그(--coverage) 및 링커 설정 변경 필요 → 팀 전체 환경 세팅 밀착 작업 부담
- 커버리지 측정 하나를 위해 빌드 시스템을 건드리는 것은 리스크 대비 투자 비용이 큰 작업

해결

AI를 활용해 환경 셋업 및 커버리지 측정 전 과정을 자동화

- 컴파일러 플래그·링커 설정·실행 순서를 AI와 함께 구성 → 빌드 스크립트 자동 생성
- line / function / branch 세 지표를 한 번에 측정·리포트 출력하는 스크립트 시스템화

TC 현황 & Coverage



측정 도구: llvm-cov | 대상: 전체 소스 (테스트 파일 제외)



05 회고



회고

✓ 잘된 점

TDD로 각 레이어 독립 검증
→ 통합 시 충돌 0건

코드 리뷰로 포터빌리티
버그 조기 발견

역할 분리로 병렬 개발
→ 일정 단축

⚠ 어려웠던 점

TDD vs. non-TDD의 차이를 느낄 수 있었다.

Test Coverage 확보를 위한 TC 추가가 TDD
개발에서 훨씬 수월했던 거 같다.

🔮 다음엔

인터페이스 설계를 팀이
함께 사전에 확정

CI 파이프라인 도입
(빌드 + 테스트 자동화)

Q & A

감사합니다

github.com/jiyeun7-Lee/CodeFabInterpreter

PR 리뷰 참여도 — Inline Comments per PR

Inline Code Review Comments per PR

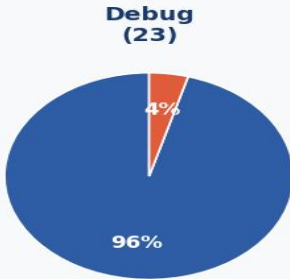
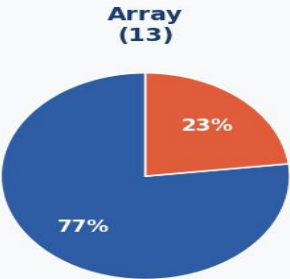
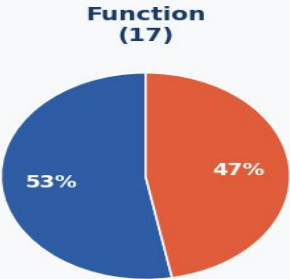
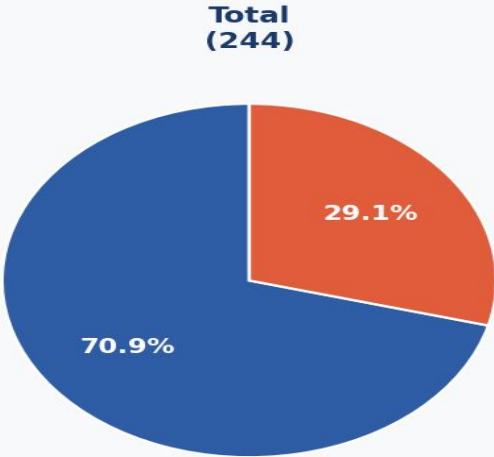
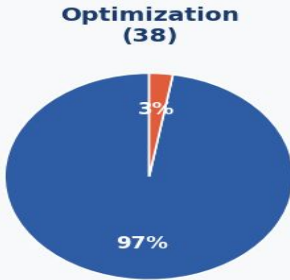
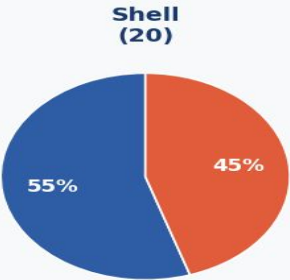
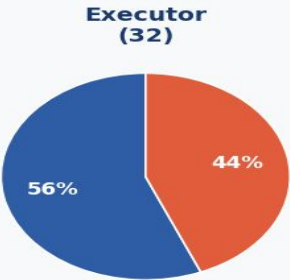
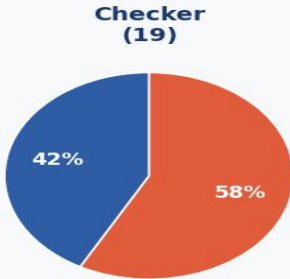
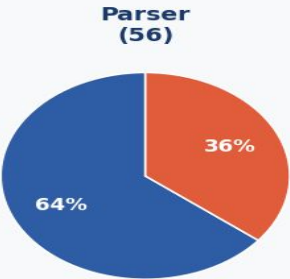
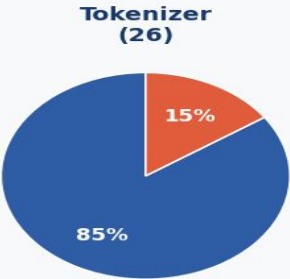


* 전체 PR 대상 (open + closed) | 데이터 기준: GitHub Pull Request inline review comments

Appendix A — Inline Review Comment가 없는 PR 목록

PR#	Author	제목	주요 변경
#1	jiyun7-Lee	Rename CodeFab_README.md to README.md	README 파일명 변경
#2	hahagil	Add the textbook for project	교재 파일 추가
#3	hahagil	(fix) Update assignee	담당자 변경 (김홍원→송용길)
#8	mingusim	[feature] Checker 구현	중복 선언 감지, 자기 초기화 참조, 블록 스코프 스택
#11	jiyun7-Lee	[Feature] ExpressionStmt 실행 지원 및 TC 추가	executeStatement()에 ExpressionStmt 분기 추가
#12	epicpoem	[Feature] TC추가 및 기능 구현	누락 TC 추가 및 관련 기능 구현
#13	hahagil	[feat] Tokenizer TC-14~20 구현 및 에러 처리 추가	미종결 문자열·알 수 없는 문자 runtime_error throw
#14	jiyun7-Lee	ExpressionStmt 실행 및 UnaryExpr/GroupingExpr 추가	UnaryExpr/GroupingExpr evaluate 구현
#17	mingusim	[refactor] Checker 호출 간 스코프 유지	scopes_ 멤버 승격, TEST_F Fixture 전환
#21	hahagil	Bugfix/utf8	UTF-8 콘솔 출력 버그 수정
#23	hahagil	[fix] Apply SetConsoleOutputCP to Debug build	Debug 빌드에 SetConsoleOutputCP(CP_UTF8) 추가
#24	epicpoem	[Doc] Custom Language 사용법 README 추가	README 언어 사용법 명시, spec 파일 정리
#34	epicpoem	[fix] ForStmt 스코프 격리 및 에러 메시지 개선	ForStmt init 변수 상위 스코프 누출 버그 수정
#37	hahagil	[unitTest] Add missing TCs for issues #28-#33	Tokenizer/Parser/Executor/Checker/Shell/Function TC 추가
#40	epicpoem	[doc] spec_korean.txt 이미지 내용 포함 업데이트	spec 문서 이미지 내용 텍스트로 통합
#44	mingusim	[unitTest] 정적 바인딩·상수 폴딩 TC 추가	MockEnvironment, countBinaryExpr, TC 명세 전면 재작성
#50	jiyun7-Lee	[Doc] readme 수정	938줄→170줄 정리, FactoryShell/Debug 명령어 추가
#53	jiyun7-Lee	[refactor] UnitTestShell	BaseRunner 추출, BraceContext, TestHelpers 공통화

Appendix B — TC Positive / Negative 비율 (총 244개)



Positive (173) Negative (71)

Appendix C — TDD Best Practice Top 10

PR	Author	TDD 패턴	핵심 스토리
#5	hahagil	명세 선작성 + Red→Green 커밋 분리	TC_Tokenizer.md(349줄) 먼저 작성 → TC-01~12 전부 Red → tokenize() 구현 후 일괄 Green. PR body가 Red/Green 섹션으로 명시적 분리된 교과서적 TDD.
#16	epicpoem	TC가 인터페이스 설계 결정	runLine() stub만 있는 상태에서 TC-PS-01~05 선작성(8 FAILED). TC-PS-03이 executor_를 멤버로 유지하는 설계를, TC-PS-04/05가 에러 처리 흐름을 강제.
#22	epicpoem	TC가 Dangling Pointer 버그 사전 발견	TC-PS-09(한 runLine 선언 → 다음 runLine 호출) Red → FunctionValue::body raw ptr dangling. TC가 unique_ptr 소유권 이전 fix를 이끌고 런타임 크래시 차단.
#25	hahagil	Negative TC 31개 → 리팩토링 안전망	7개 모듈 31개 Negative TC 추가. 음수 인덱스(A-23), Checker 에러 후 상태 보존(S-31) 등 implicit 가정 명문화. PR #47·#48 리팩토링 시 회귀 방지망으로 작동.
#37	hahagil	이슈→TC→프로덕션 3파일 연쇄 수정	이슈 #28~#33 TC 14개 선작성. TC-FN-16 하나가 Stmt.h / Parser.cpp / Checker.cpp 3파일 연쇄 수정을 유발. TC가 버그 티켓을 실행 가능한 명세로 변환.
#13	hahagil	[test]→[feat]→[refactor] 커밋 3단 분리	TC-14/16(미종결 문자열·알 수 없는 문자)이 silent-skip을 Red로 만들어 구현 4줄 수정 유도. 커밋 메시지 자체가 TDD 3스텝으로 작성됨.
#9	jiyun7-Lee	[test]→[feat] 교차 4회, TC10이 C++ 함정 차단	[test]→[feat] 쌍 4회 반복. TC10 DivideByZero가 double/0=inf라는 C++ 묵인 동작을 명시적 차단으로 강제. 인크리멘탈 TDD 리듬이 커밋 이력에 기록됨.
#6	mingusim	895줄 TC 명세 선작성 → 인터페이스 결정	TC_StmtParserAndChecker.md(895줄)가 구현 전 먼저 커밋. 12개 리뷰에서 TC 기준으로 시그니처·네이밍 충돌 지적 → FakeExprParser 구조 탄생.
#34	epicpoem	ForScope TC가 스코프 누출 버그 발견	ForScope_InitVarNotLeaking TC가 for 변수 외부 누출 버그를 Red로 포착 → forEnv 추가. 에러 메시지 형식 TC 3개가 한국어 줄번호 스펙을 구현이 따르도록 강제.
#36+#44	mingusim	TC가 의미론 버그 발견 + 성능 요건 명세	CF-TC 패턴D가 x*0 사이드 이펙트 소실 버그(Red) → isPure 가드 추가. PR #44에서 countBinaryExpr로 폴딩 5→0 수치 검증, gmock으로 O(1) 성능 명세.
총 285개 TC (Positive 173 / Negative 71+) 전체 PR 중 TDD 흔적 명확한 PR 10개 선정			

Appendix D — PR 코드 리뷰 데이터

Inline Comments Top 5

#4 24 epicpoem nullptr→크래시, new 누수 → make_unique 6파일 리팩

#38 12 jiyun7-Lee depth_ RAII 부재, FunctionCallExpr depth_ 미증가

#22 12 epicpoem dangling ptr, REPL 경계 넘는 함수 호출 버그

#6 12 mingusim 895줄 TC 명세 기반 인터페이스 설계 결정

#5 12 hahagil TC 명세 선작성 + Red→Green 커밋 분리 TDD

잠재 버그 발견 Top 5

#4 hahagil nullptr→Executor 크래시
new 누수 → make_unique 리팩

parsePrimary() nullptr → makeBinary → Executor 역참조 크래시. 6파일 unique_ptr 전환 유도.

#38 epicpoem depth_ RAII 없음 → 영구 증가
FuncCall depth_ 미증가

ReturnSignal throw 시 depth_-- 스킵 → 호출 깊이 영구 누적. DebugShell next 버그도 추가 발견.

#26 epicpoem getAt nullptr 가드 없음
const& 시그니처·AST 변경 불일치

distance > 환경 체인 깊이 시 nullptr 역참조 UB/크래시. const& 시그니처인데AST 노드 실제 수정.

#20 epicpoem check() 재호출 시 scopes_ 미초기화
FakeExprParser 제거로 테스트 독립성 파괴

동일 Checker 인스턴스 check() 두 번 호출 시 scopes_ 누적 → false positive 중복 오류.

#43 mingusim brace depth 계산 시
문자열 내 {} 미필터

var s = "{ nested }"; 문자열 내부 {}가 depth 계산에 포함 → 멀티라인 누적 오동작.

Appendix E — Issue → PR 해결 트래킹

